

Up To Speed — EAI Rigged Rock-Paper-Scissors Project

This document walks through every file you worked on or touched, in the order that makes the most logical sense. By the end you should be able to explain what every line does, why it's there, and how everything connects.

It covers:

- What TFLite is and why you need it
 - What MobileNet is and why you use it instead of training from scratch
 - Every function in both training scripts with what each line does and why
 - The Makefile changes and how Make's dependency tracking works
 - All four .h files — what they do, why they're separate, and how they connect to the Python side
 - TfliteImageClassifier — how it loads, quantizes, and runs inference
 - A full data flow diagram from camera frame to final decision
 - A quick-reference table with all the numbers you'll need for the presentation
-

The Big Picture First

The project is a rigged Rock-Paper-Scissors game running on a Raspberry Pi 3.

- The Pi has a camera that looks at the player's hand.
- The Pi runs **two AI models** to make decisions:
 1. **Gesture model** — looks at the hand and says "that's rock", "that's paper", "that's scissors", or "neutral" (hand not in position yet).
 2. **Accessory model** — looks at the hand and says "wearing the accessory" or "not wearing it".
- If the player **has the accessory** → the Pi deliberately picks a gesture it will **lose** with.
- If the player **has no accessory** → the Pi picks a gesture it will **win** with.
- The Pi shows its choice on the LED matrix, then the full screen goes green (player wins) or red (player loses).

You built the **AI side** of this: training both models in Python, converting them to a format the Pi can run, and writing the C++ headers that connect the models to the rest of the C++ code.

Concept: What is TFLite?

Training an AI model in Python (with TensorFlow/Keras) produces a large file with millions of numbers. A Raspberry Pi 3 has very limited memory and no GPU — it can't run that full model.

TensorFlow Lite (TFLite) is a stripped-down version of TensorFlow designed for small devices. The conversion process:

1. Takes your trained Keras model.
2. Strips out everything only needed for training (optimizers, gradients, etc.).

3. Applies **quantization** — reduces the precision of the numbers from 32-bit floats to 8-bit integers (`int8`). This makes the model ~4× smaller and faster on a CPU with no real accuracy loss.

The result is a `.tflite` file — a single compact file the C++ code on the Pi loads and runs.

Concept: What is MobileNet?

Training a vision model from zero would require millions of images and days of compute. Instead, you use **transfer learning**:

- **MobileNet** is a model pre-trained by Google on 1.4 million images (ImageNet). It already "knows" how to detect edges, shapes, textures.
- You take MobileNet's learned layers (the **backbone**), freeze them so they don't change, add a small new **head** (a few layers of your own), and train only the head on your much smaller dataset.
- This is **Phase 1** of training.
- Then in **Phase 2** (fine-tuning), you unfreeze the last few backbone layers and train them too at a very low learning rate, so they adapt slightly to your specific images.

You use two variants:

- **MobileNetV2** for the gesture model — classic, well-tested, input range `[-1, 1]`
- **MobileNetV3Small** for the accessory model — newer, smaller, slightly faster, input range `[0, 1]`

The input range difference is critical — it must match exactly between Python training and C++ inference. More on this later.

File 1: `requirements.txt`

```
tensorflow==2.16.1
tf-keras==2.16.*
numpy==1.26.4
tensorflow-model-optimization==0.8.0
```

This is the list of Python libraries the training scripts need. The `Makefile` uses it to set up the virtual environment (`.venv/`) automatically. You never edit this by hand during normal work — `make train` handles installing everything.

File 2: `src/train_gesture.py`

This script trains the gesture classifier (rock / paper / scissors / neutral) and exports it as a TFLite model.

The constants at the top

```
CLASS_NAMES = ["neutral", "paper", "rock", "scissors"]
NUM_CLASSES = len(CLASS_NAMES)
```

```
IMAGE_SIZE = (224, 224)
BATCH_SIZE = 4
```

- `CLASS_NAMES` — the four possible gestures. The order is **alphabetical** because TensorFlow's `image_dataset_from_directory` assigns label numbers alphabetically. So `neutral=0`, `paper=1`, `rock=2`, `scissors=3`. This order **must** match `model_classes.h` in C++.
- `IMAGE_SIZE` — MobileNetV2 expects 224×224 pixel images. The camera is also set to capture 224×224.
- `BATCH_SIZE = 4` — the model sees 4 images at a time during training. Small batch because the dataset is small.

parse_args()

```
def parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(...)
    p.add_argument("--data-dir", default="data")
    ...
```

This lets you pass options when running the script from the command line:

```
python src/train_gesture.py --epochs 30 --data-dir mydata/
```

If you don't pass anything, the defaults are used (`data/` folder, 20 epochs, etc.).

load_dataset()

```
full_ds = tf.keras.utils.image_dataset_from_directory(
    str(data_dir),
    image_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=True,
    seed=seed,
    label_mode="int",
)
```

This scans the `data/` folder which has subfolders `neutral/`, `paper/`, `rock/`, `scissors/`. Each image file becomes a training example. The label is automatically the folder name, turned into an integer (alphabetical order = 0,1,2,3).

`label_mode="int"` means labels are integers (0,1,2,3), not one-hot vectors.

```
found_classes = full_ds.class_names
if found_classes != CLASS_NAMES:
```

```
raise ValueError(...)
```

This is a safety check. If the folder names don't match exactly what we expect, the label numbers would be wrong and the model would learn garbage silently. This makes the script crash with a clear error instead.

make_pipelines()

Splits the data into three sets:

- **train_ds** — used to actually update the model's weights
- **val_ds** (validation) — used to monitor progress during training, never used to update weights
- **test_ds** — held out completely, only used once at the very end to measure real accuracy

The split is ~70% train / 15% val / 15% test, calculated in batches.

Augmentation — the training set gets randomly modified to make the model more robust:

```
augment = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomBrightness(0.15),
])
```

- **RandomFlip** — randomly mirrors images left-right. A rock sign mirrored is still a rock sign.
- **RandomBrightness** — randomly makes images slightly brighter or darker. Helps the model handle different lighting.

Validation and test sets are **not** augmented — you always evaluate on clean data so your accuracy numbers are honest.

Preprocessing (the critical part):

```
def preprocess(images, labels):
    images = tf.cast(images, tf.float32)
    images = (images / 127.5) - 1.0
    return images, labels
```

Camera pixels come in as integers 0–255. MobileNetV2 was trained with pixels in the range [-1, 1]. This formula converts:

- $0 \rightarrow (0/127.5) - 1.0 = -1.0$
- $255 \rightarrow (255/127.5) - 1.0 = +1.0$

This exact formula must be replicated in the C++ code (preprocess.h). If it doesn't match, the model receives numbers it was never trained on and produces garbage predictions.

`.prefetch(tf.data.AUTOTUNE)` — while the GPU/CPU is processing one batch, the next batch is already being loaded in the background. Speeds up training.

`make_model()`

```
base = tf.keras.applications.MobileNetV2(
    input_shape=(*IMAGE_SIZE, 3),
    include_top=False,
    weights="imagenet",
)
base.trainable = False
```

- Downloads MobileNetV2 pre-trained on ImageNet.
- `include_top=False` — removes MobileNet's own final classification layer (it classifies 1000 ImageNet categories, we don't want that).
- `weights="imagenet"` — use Google's pre-trained weights.
- `base.trainable = False` — freeze all backbone layers for Phase 1.

```
inputs = tf.keras.Input(shape=(*IMAGE_SIZE, 3), name="image_input")
x = base(inputs, training=False)
x = tf.keras.layers.GlobalAveragePooling2D(name="gap")(x)
x = tf.keras.layers.Dense(128, activation="relu", name="dense1")(x)
x = tf.keras.layers.Dropout(0.3, name="dropout")(x)
outputs = tf.keras.layers.Dense(NUM_CLASSES, activation="softmax",
name="output")(x)
```

The new **head** layers added on top of the frozen backbone:

- `GlobalAveragePooling2D` — MobileNetV2's backbone outputs a 3D feature map (7×7×1280). This collapses it to a flat 1280-number vector by averaging.
- `Dense(128, relu)` — a fully connected layer with 128 neurons, learns combinations of the 1280 features.
- `Dropout(0.3)` — during training, randomly switches off 30% of neurons each step. Prevents the model from memorising the training data (overfitting).
- `Dense(NUM_CLASSES, softmax)` — final layer, 4 outputs (one per gesture). `softmax` converts raw scores to probabilities that sum to 1.0.

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

- `Adam` — an optimizer that adjusts the learning rate automatically per parameter. Standard choice.

- `learning_rate=1e-3` — step size for weight updates (0.001). Higher for Phase 1 because we're only training the head.
- `sparse_categorical_crossentropy` — the right loss function for multi-class classification where labels are integers (not one-hot).

`unfreeze_top_layers()`

```
def unfreeze_top_layers(model: tf.keras.Model, n_unfreeze: int = 30) ->
None:
    base = model.get_layer("mobilenetv2_1.00_224") # fixed bug
    base.trainable = True
    for layer in base.layers[:-n_unfreeze]:
        layer.trainable = False
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
        ...
    )
```

Phase 2: unfreeze the **last 30 layers** of the MobileNetV2 backbone. The early layers detect basic things (edges, corners) — we keep those frozen. The later layers detect complex things (shapes specific to the training data) — we let those adapt.

`learning_rate=1e-5` (0.00001) — 100× smaller than Phase 1. We don't want to destroy the pre-trained knowledge, just nudge it.

The bug that was here: it said `model.get_layer("MobileNetV3Small")` — wrong name for a MobileNetV2 model. This would crash before Phase 2 could run. Fixed to `"mobilenetv2_1.00_224"`.

`make_callbacks()`

```
tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=6,
restore_best_weights=True)
tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,
patience=3)
```

Callbacks fire automatically during training:

- `EarlyStopping` — if validation loss hasn't improved for 6 epochs in a row, stop training early and restore the best weights seen. Prevents overfitting and saves time.
- `ReduceLROnPlateau` — if validation loss hasn't improved for 3 epochs, cut the learning rate in half. Helps squeeze out a bit more accuracy when progress stalls.

`export_tflite()`

```
model.save(str(keras_path))          # save full Keras model
model.export(str(saved_model_dir))   # export as SavedModel format
```

Two saves: the `.keras` file lets you reload and keep training later. The `SavedModel` directory is the format TFLite converts from.

```
converter = tf.lite.TFLiteConverter.from_saved_model(str(saved_model_dir))
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
tflite_bytes = converter.convert()
```

This is the **int8 quantization** step:

- `Optimize.DEFAULT` — tells the converter to quantize.
- `representative_dataset` — feeds 40 real images through so the converter can measure the actual range of values at each layer and choose good int8 scale factors.
- `TFLITE_BUILTINS_INT8` — use only int8 ops (fastest on Pi's CPU).
- `inference_input_type = tf.int8 / inference_output_type = tf.int8` — the model's input and output tensors are int8, not float32. The C++ code handles converting floats to int8 before feeding in.

Result: `artifacts/gesture_model.tflite` — ~2.8 MB, 95% test accuracy.

`main()`

Calls everything in order:

1. Load dataset → build pipelines → build model → Phase 1 training → Phase 2 fine-tuning → evaluate on test set → export TFLite → save metrics CSV.

File 3: `src/train_accessory.py`

Almost identical structure to `train_gesture.py`. Key differences:

Binary classification (2 classes instead of 4)

```
CLASS_NAMES = ["absent", "present"]
NUM_CLASSES = 2
```

`absent=0` (no accessory), `present=1` (wearing accessory). Alphabetical again.

MobileNetV3Small instead of MobileNetV2

```
base = tf.keras.applications.MobileNetV3Small(  
    include_preprocessing=False,  
    ...  
)
```

- MobileNetV3Small is a newer, lighter model — good for binary classification where MobileNetV2 would be overkill.
- `include_preprocessing=False` — MobileNetV3 normally includes its own preprocessing layer. We turn it off because we handle preprocessing ourselves (so the C++ code can replicate it).

Different preprocessing formula

```
images = images / 255.0    # maps [0,255] → [0.0, 1.0]
```

MobileNetV3 expects `[0, 1]`, not `[-1, 1]`. This is different from the gesture model. **This difference must be reflected in C++** — which is why there are two separate preprocessing headers.

More augmentation

```
augment = tf.keras.Sequential([  
    tf.keras.layers.RandomFlip("horizontal"),  
    tf.keras.layers.RandomBrightness(0.2),  
    tf.keras.layers.RandomContrast(0.1),  
])
```

Extra `RandomContrast` compared to the gesture model. Accessories can be subtle — more augmentation helps the model generalise across different lighting and contrast conditions.

Smaller head

```
x = tf.keras.layers.Dense(64, activation="relu", name="dense1")(x)
```

Only 64 neurons instead of 128 — binary classification is simpler, doesn't need as many.

Higher accuracy bar

```
if acc < 0.90:  
    print("WARNING: accuracy below 90%...")
```

Gesture model warns at 85%. Accessory model warns at 90% — being wrong about the accessory directly breaks the rigging mechanic, so we hold it to a higher standard.

Result: `artifacts/accessory_model.tflite` — ~1.2 MB, 99.4% test accuracy.

File 4: `Makefile` (the changes)

The Makefile is a build script. You run `make <target>` and it executes the right commands.

What was there before (broken)

```
MODEL_FILE := $(ARTIFACT_DIR)/model.tflite
$(MODEL_FILE): scripts/train_and_convert.py ...
    "$(PYTHON_BIN)" scripts/train_and_convert.py ...
train: $(MODEL_FILE)
```

This pointed to a single script `scripts/train_and_convert.py` that **doesn't exist**. Running `make train` would crash immediately.

What it is now (fixed)

```
GESTURE_MODEL := $(ARTIFACT_DIR)/gesture_model.tflite
ACCESSORY_MODEL := $(ARTIFACT_DIR)/accessory_model.tflite

$(GESTURE_MODEL): src/train_gesture.py requirements.txt $(VENV_STAMP) |
$(ARTIFACT_DIR_STAMP)
    "$(PYTHON_BIN)" src/train_gesture.py --artifacts-dir "$(ARTIFACT_DIR)"
$(TRAIN_ARGS)

$(ACCESSORY_MODEL): src/train_accessory.py requirements.txt $(VENV_STAMP) |
$(ARTIFACT_DIR_STAMP)
    "$(PYTHON_BIN)" src/train_accessory.py --artifacts-dir
"$(ARTIFACT_DIR)" $(TRAIN_ARGS)

train: $(GESTURE_MODEL) $(ACCESSORY_MODEL)
```

- `$(GESTURE_MODEL)` is a Make variable pointing to `artifacts/gesture_model.tflite`.
- The indented line is the shell command that runs if the file doesn't exist yet (or if `train_gesture.py` changed).
- `train: $(GESTURE_MODEL) $(ACCESSORY_MODEL)` means "to do `make train`, ensure both model files exist."
- `$(VENV_STAMP)` — Make first ensures the Python virtual environment is set up.

How Make's dependency tracking works: Make only re-runs a step if the output file is older than its inputs. So if you run `make train` twice without changing anything, the second run does nothing — the `.tflite` files are newer than the `.py` files.

File 5: `src/preprocess.h`

This is a C++ header file. A `.h` (header) file declares functions and constants that other C++ files can use by `#include`-ing it.

```
#pragma once
```

Ensures this file is only included once per compilation, even if multiple files try to include it.

```
namespace preprocess {
constexpr int IMAGE_WIDTH    = 224;
constexpr int IMAGE_HEIGHT   = 224;
constexpr int IMAGE_CHANNELS  = 3;
constexpr int IMAGE_ELEMENTS = IMAGE_WIDTH * IMAGE_HEIGHT * IMAGE_CHANNELS;
// 150528
}
```

`constexpr` = compile-time constant. `IMAGE_ELEMENTS` is the total number of numbers in one flattened image: $224 \times 224 \times 3 = 150,528$.

```
inline std::vector<float> frame_to_float(const rpica::RgbFrame& frame)
{
    std::vector<float> out;
    out.reserve(IMAGE_ELEMENTS);
    for (std::size_t i = 0; i < frame.rgb.size(); ++i) {
        out.push_back(
            static_cast<float>(frame.rgb[i]) / 127.5f - 1.0f
        );
    }
    return out;
}
```

Takes a raw camera frame (pixels as `0–255` bytes) and returns a `std::vector<float>` of 150,528 values in `[-1, 1]`.

This is the C++ version of the Python line:

```
images = (images / 127.5) - 1.0
```

They must be identical. If you change one, you must change the other.

`inline` means the function body is pasted directly at each call site rather than being a separate function call — faster for a tight loop.

File 6: `src/preprocess_accessory.h`

Same idea as `preprocess.h` but for the accessory model (MobileNetV3, `[0, 1]` range):

```
out.push_back(static_cast<float>(frame.rgb[i]) / 255.0f);
```

C++ version of:

```
images = images / 255.0
```

Both models receive the **same camera frame**. But before passing it to each model, different preprocessing is applied. This is why there are two separate headers — you cannot use `preprocess.h` for the accessory model or vice versa; the numbers would be wrong and the model would output garbage.

File 7: `src/model_classes.h`

This file is the shared vocabulary between the trained models and the C++ game logic. It defines what the output numbers mean.

```
namespace gesture {  
  
constexpr int NEUTRAL    = 0;  
constexpr int PAPER     = 1;  
constexpr int ROCK      = 2;  
constexpr int SCISSORS  = 3;  
constexpr int NUM_GESTURE_CLASSES = 4;  
  
constexpr int ACCESSORY_ABSENT = 0;  
constexpr int ACCESSORY_PRESENT = 1;  
constexpr int NUM_ACCESSORY_CLASSES = 2;
```

These numbers match the Python `CLASS_NAMES` order exactly. When the gesture model outputs `2`, it means rock. This is where the game logic code reads `gesture::ROCK` instead of a magic number `2`.

```
constexpr const char* GESTURE_MODEL_PATH = "gesture_model.tflite";  
constexpr const char* ACCESSORY_MODEL_PATH = "accessory_model.tflite";
```

File paths to the model files on the Pi. The C++ code uses these to load the models.

```
inline int winningCounterFor(int player_gesture) {  
    switch (player_gesture) {  
        case ROCK:      return PAPER;  
        case PAPER:     return SCISSORS;
```

```

        case SCISSORS: return ROCK;
        default:       return ROCK;
    }
}

inline int losingCounterFor(int player_gesture) {
    switch (player_gesture) {
        case ROCK:      return SCISSORS;
        case PAPER:     return ROCK;
        case SCISSORS:  return PAPER;
        default:        return SCISSORS;
    }
}

```

Game logic helpers. If the player shows rock:

- `winningCounterFor(ROCK)` → `PAPER` (Pi picks paper to beat the player — player loses)
- `losingCounterFor(ROCK)` → `SCISSORS` (Pi picks scissors to lose to the player — player wins)

The game logic in `main.cpp` will call these based on the accessory model result:

- `ACCESSORY_ABSENT` → call `winningCounterFor` (Pi wins)
- `ACCESSORY_PRESENT` → call `losingCounterFor` (Pi loses, player wins)

File 8: `src/TfliteImageClassifier.h` and `.cpp`

This is the C++ class that loads a `.tflite` file and runs inference (makes predictions). It's generic — the same class is used for both models.

The header (`.h`) — the interface

```

struct ImagePrediction {
    int    predicted_class = -1;
    float  confidence      = 0.f;
    std::vector<float> scores;
};

```

The return type of a prediction. `predicted_class` is the index (e.g. 2 for rock), `confidence` is the probability score (e.g. 0.94), `scores` is the full list of probabilities for all classes.

```

class TfliteImageClassifier {
public:
    explicit TfliteImageClassifier(const std::string& model_path, int
num_classes);
    ImagePrediction Predict(const std::vector<float>& input_data);
    bool ok() const { return ok_; }
}

```

```
};
...
};
```

Usage from the game code will look like:

```
TfliteImageClassifier gesture_clf("gesture_model.tflite", 4);
TfliteImageClassifier accessory_clf("accessory_model.tflite", 2);

auto frame_floats = preprocess::frame_to_float(frame);
auto pred = gesture_clf.Predict(frame_floats);
// pred.predicted_class is now 0,1,2,3
```

The implementation (.cpp)

Load() — loads the `.tflite` file, builds the TFLite interpreter, validates the input/output tensor shapes. If the model file doesn't match expectations (wrong number of classes, wrong image dimensions), it fails with a clear error message instead of crashing silently.

CopyInput() — the model was exported as int8, so its input tensor expects `int8` values, not `float`. This function **quantizes** the floats before feeding them in:

```
int8_t QuantizeInt8(float value, float scale, int zero_point) {
    int q = std::lround(value / scale + zero_point);
    // clamp to [-128, 127]
    return static_cast<int8_t>(q);
}
```

`scale` and `zero_point` are stored inside the model itself (determined during the TFLite conversion step in Python).

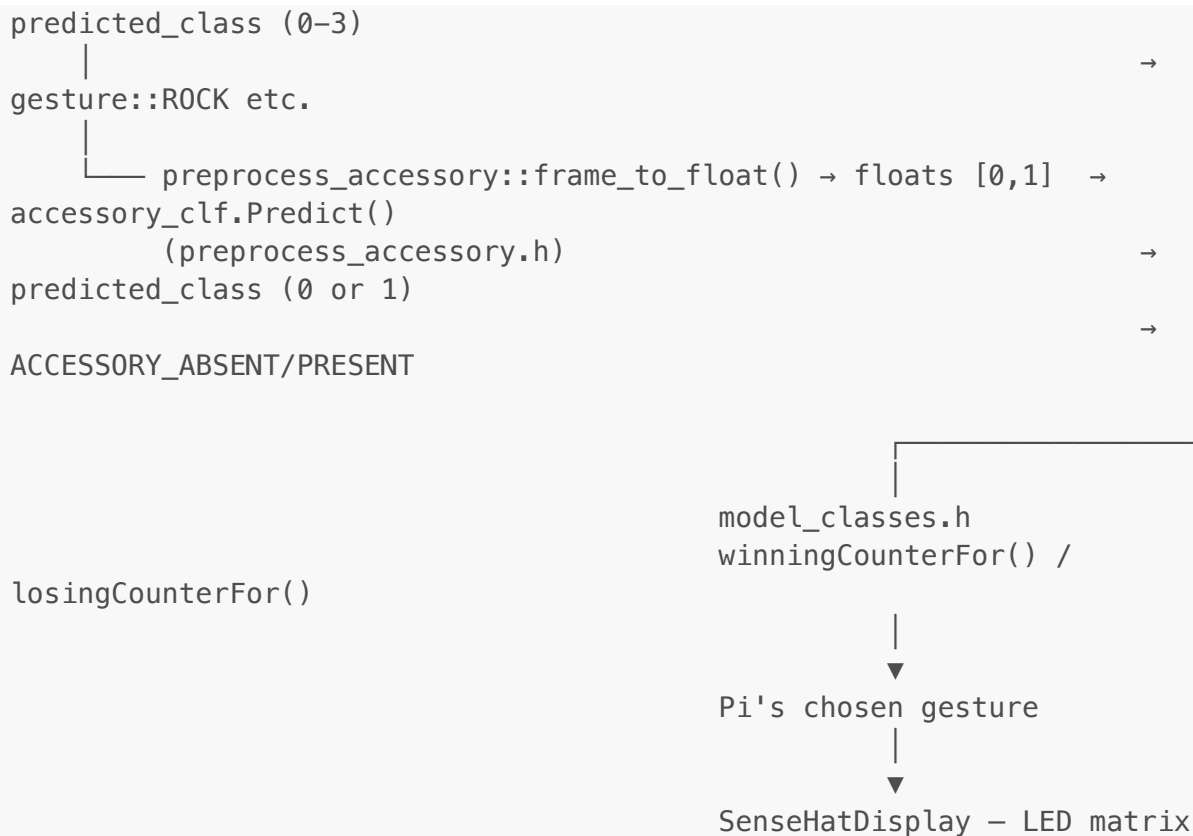
ReadOutput() — after inference, the output tensor is also int8. This **dequantizes** it back to floats (probabilities).

Predict() — the public method. Calls CopyInput → Invoke (runs the model) → ReadOutput → finds the highest-scoring class with `std::max_element`.

How Everything Connects

Here is the full data flow from camera to decision:

```
Camera (Pi) → raw RGB frame [224×224×3, values 0–255]
    |
    |—— preprocess::frame_to_float() → floats [-1,1] →
gesture_clf.Predict()
    |
    | (preprocess.h) →
```



Where the models live and how they get there

Training happens in the **devcontainer** (your laptop/desktop environment):

1. `make train` → runs both Python scripts → produces `artifacts/gesture_model.tflite` and `artifacts/accessory_model.tflite`
2. `make deploy` → copies the built C++ binary and the `.tflite` files to the Pi over SSH

The Pi runs the C++ binary which loads both `.tflite` files at startup.

Quick Reference: Numbers to Know for the Presentation

	Gesture Model	Accessory Model
Architecture	MobileNetV2	MobileNetV3Small
Classes	4 (neutral/paper/rock/scissors)	2 (absent/present)
Input preprocessing	pixel/127.5 - 1.0 → [-1, 1]	pixel/255.0 → [0, 1]
Quantization	int8	int8
TFLite file size	~2.8 MB	~1.2 MB
Test accuracy (Keras)	95.00%	99.37%

	Gesture Model	Accessory Model
Total model size	~4 MB (well under 100 MB limit)	
Training phases	Phase 1: head only ($lr=1e-3$) / Phase 2: top 30 backbone layers ($lr=1e-5$)	Phase 1: head only / Phase 2: top 20 backbone layers